1. **Aim/Overview of the practical:**

   Implementation of Python Basic Libraries such as

   Math

   NumPy

   Matplotlib

   Seaborn

   SciPy.

   2. Creation and loading different Datasets in Python.

   3. Write a program to implement Linear regression.

   Download different datasets from Kaggle or UCI ML Repository.

   Import all the necessary modules for the datasets.

   Find out the best fit line along with the MSE, RMSE.

2. **Objective of the practical:** To implement and demonstrate basic Python libraries and linear regression using real-world datasets for data analysis and visualization.

3. **Code for experiment/practical:**

(A) Python Basic Libraries

```
# Importing required libraries
import math
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# math library
print(" Math")
angle = math.pi / 4
print(f"Factorial of 5 = {math.factorial(5)}")
print(f"Cos({angle}) = {math.cos(angle)}")
print(f"Square root of 16 = {math.sqrt(16)}")
print(f"Logarithm base e of 10 = {math.log(10)}")

#  numpy
print("NumPy")
data = np.array([1, 2, 3, 4])
print(f"Array: {data}")
print(f"Mean: {np.mean(data)}")
print(f"Standard Deviation: {np.std(data)}")
print(f"Median: {np.median(data)}\n")

# matplotlib
print(" Matplotlib Plot")
x = np.linspace(0, 30, 200)
y = np.sin(x)
plt.figure(figsize=(4, 2))
plt.plot(x, y, label='sin(x)')
plt.title('Sine Wave')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend()
plt.grid(True)
plt.show()

#  seaborn
print( "Seaborn Plot")
tips = sns.load_dataset("tips")
```

```
plt.figure(figsize=(4, 2))
sns.boxplot(x="day", y="total_bill", data=tips)
plt.title("Total Bill by Day")
plt.show()

# scipy
print(" SciPy Statistical Test")
group1 = np.random.normal(10, 15, 150)
group2 = np.random.normal(20, 55, 150)
t_stat, p_val = stats.ttest_ind(group1, group2)
print(f"T-statistic: {t_stat:.2f}, P-value: {p_val:.4f}")
if p_val < 0.05:
    print("Result: Statistically significant difference between groups.")
else:
    print("Result: No significant difference between groups.")
```

(B) Datasets

```
import numpy as np
import pandas as pd
from sklearn import datasets

# Creating dataset using NumPy
print("\nDataset using NumPy:")
np_dataset = np.random.randint(1, 100, size=(5, 3))
print(np_dataset)

# Creating dataset using pandas DataFrame
print("\nDataset using pandas DataFrame:")
data = {
    'Name': ['Varun', 'Prince', 'Ankush', 'Nikhil'],
    'Age': [20, 22, 22, 22],
    'City': ['Ambala', 'Zirakpur', 'Kharar', 'Chandigarh']
}
df = pd.DataFrame(data)
print(df)


# Loading external dataset
print("\nLoading dataset from CSV file:")
csv_df = pd.read_csv("household_power_consumption.csv")
print(csv_df.head())
```

(c) Linear Aggression:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

#  Load CSV
df = pd.read_csv("household_power_consumption.csv")

# Drop columns that are not useful for regression
df = df.drop(['Date', 'Time'], axis=1)

#  Handle missing values
df = df.apply(pd.to_numeric, errors='coerce')
df = df.dropna()

# Define X (input features) and y (target variable)
X = df.drop('Global_active_power', axis=1)
y = df['Global_active_power']

#  Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#  Train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)

#  Evaluate model using Mean Squared Error and Root Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
```

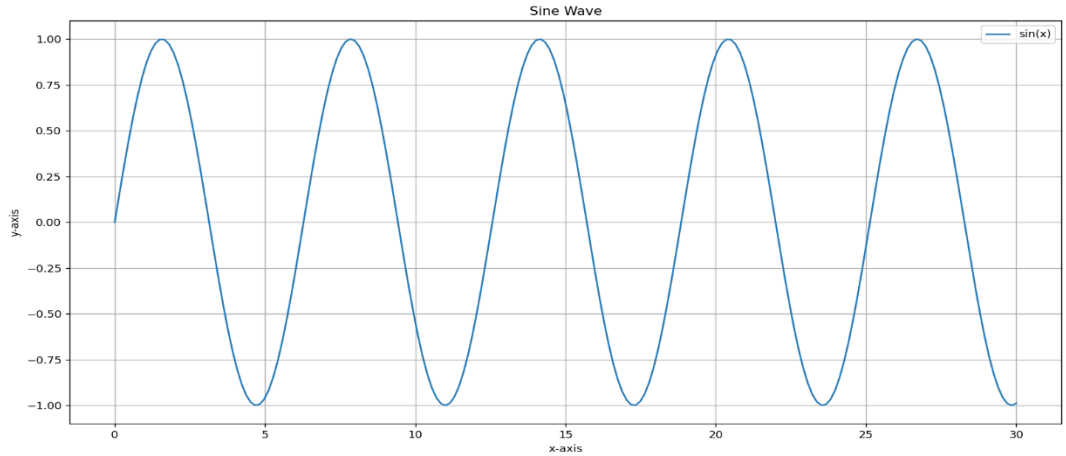
```
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)

#  Visualize Actual vs Predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.3, s=10, label="Predicted Points")
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', label="Perfect Prediction Line")
plt.xlabel("Actual Global Active Power (kilowatts)")
plt.ylabel("Predicted Global Active Power (kilowatts)")
plt.title("Actual vs Predicted - Electricity Consumption")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

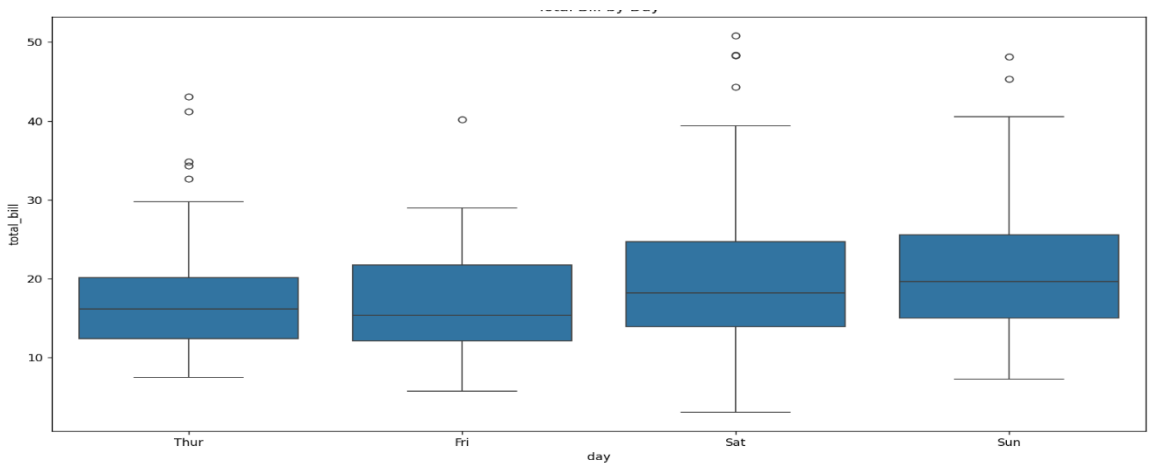
4. **Output:**

(A)

```
 Math
Factorial of 5 = 120
Cos(0.7853981633974483) = 0.7071067811865476
Square root of 16 = 4.0
Logarithm base e of 10 = 2.302585092994046
NumPy
Array: [1 2 3 4]
Mean: 2.5
Standard Deviation: 1.118033988749895
Median: 2.5
```

```
 SciPy Statistical Test
T-statistic: -3.33, P-value: 0.0010
Result: Statistically significant difference between groups.
```

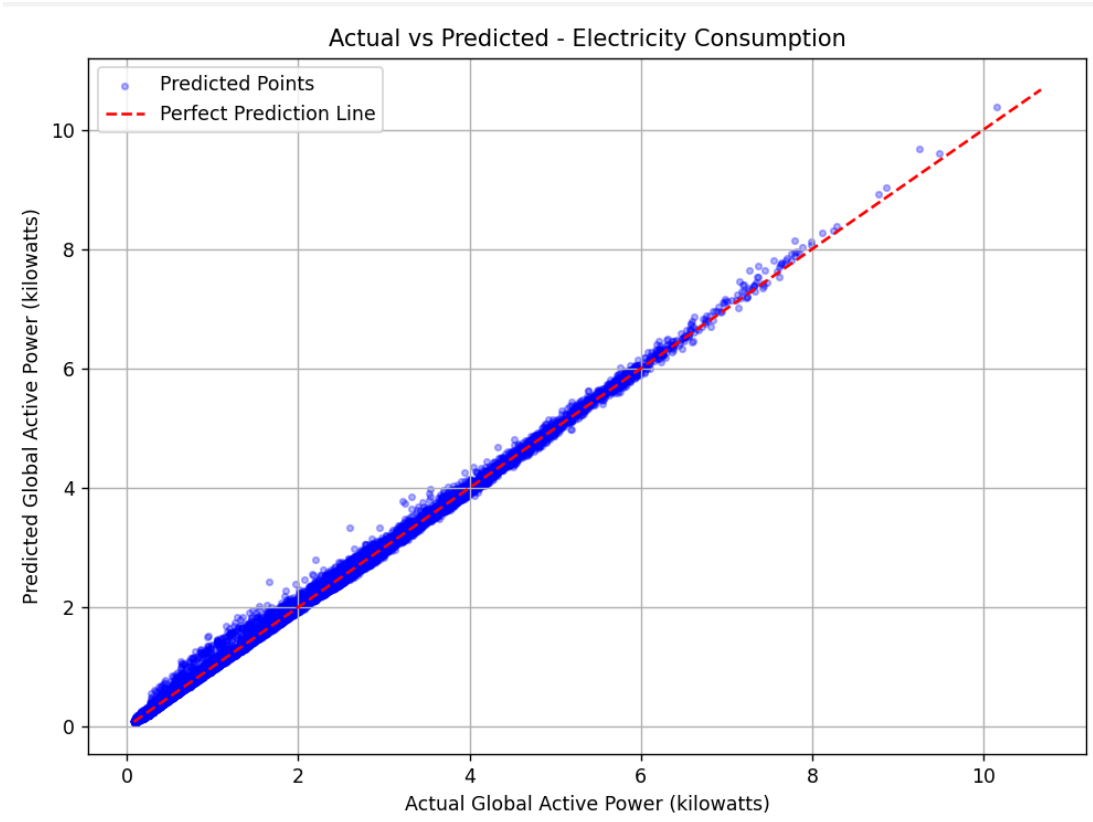




(B)

```
Dataset using NumPy:
[[57 60 18]
 [ 8 76 12]
 [96 44 16]
 [39 60 50]
 [35 29 23]]

Dataset using pandas DataFrame:
     Name  Age        City
0   Varun   20      Ambala
1  Prince   22    Zirakpur
2  Ankush   22      Kharar
3  Nikhil   22  Chandigarh

Loading dataset from CSV file:
   index    Date     Time Global_active_power Global_reactive_power Voltage Global_intensity Sub_metering_1 Sub_metering_2  Sub_metering_3
0      0  1/1/07  0:00:00               2.58                0.136  241.97            10.6             0             0             0.0
1      1  1/1/07  0:01:00              2.552                  0.1  241.75            10.4             0             0             0.0
2      2  1/1/07  0:02:00               2.55                  0.1  241.64            10.4             0             0             0.0
3      3  1/1/07  0:03:00               2.55                  0.1  241.71            10.4             0             0             0.0
4      4  1/1/07  0:04:00              2.554                  0.1  241.98            10.4             0             0             0.0
PS C:\Users\AARYAMAN\Desktop\MCA_ALL SUbjectsNOtes\SEMESTER-3\MACHINE_LEARNING\ML_Lab>
```

(C)



5. Learning Outcomes (What I have learnt):

1. To understand and implement basic Python libraries like Math, NumPy, Matplotlib, Seaborn, and SciPy.
2. To load and analyze datasets for data visualization and processing.
3. To apply linear regression on real-world datasets and evaluate performance using MSE and RMSE.